

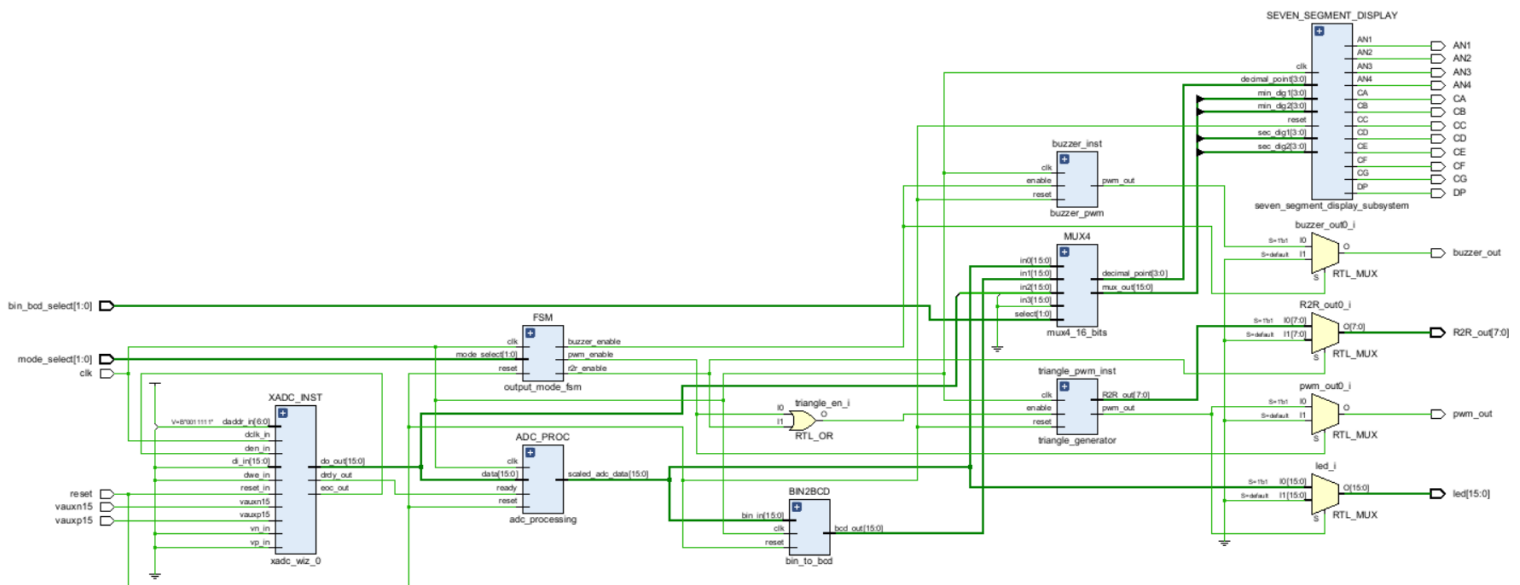
Lab 6 – PWM and R2R Waveform Generation

Introduction

In Lab 6, we will generate several waveforms using Pulse Width Modulation (PWM) and R2R ladder circuits. We will use the XADC from the previous lab project, to be able to quickly check our outputs. As for the previous lab project, you will be given a working project (see picture below) that you will recreate. Then you will make some changes to help you understand the important modules.

You may use AI to help you design, with the following requirements:

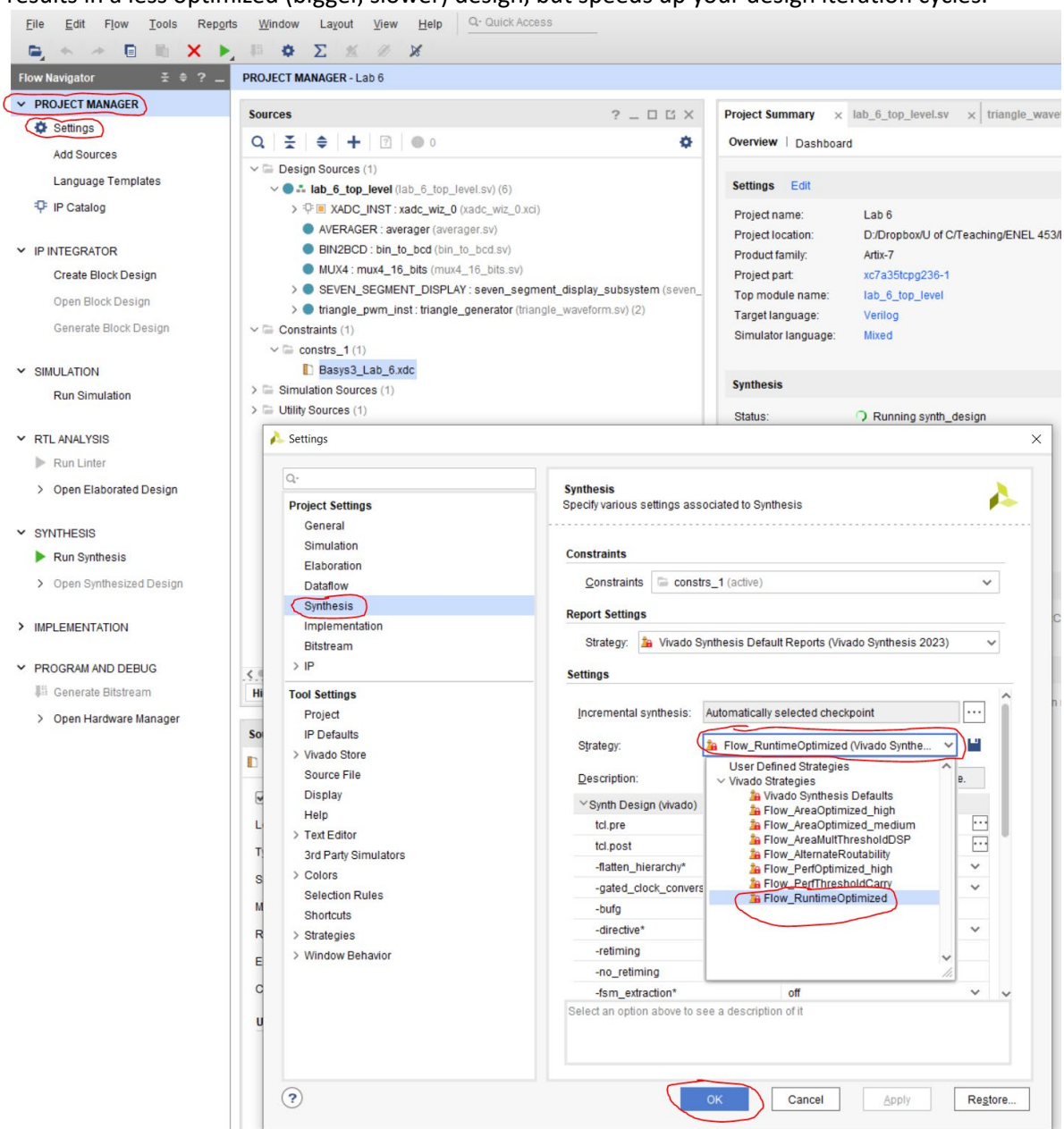
- You must thoroughly understand whatever you incorporate into your design, from the AI.
- You must include your complete AI chat transcript or transcripts into your Design Record. Include these transcripts at the end, after all the “DO” requirements. You have permission to use AI, the transcripts are simply for full disclosure and to maintain academic integrity. Do not edit or modify the transcripts, leave them “as is.” However, any code samples that the AI opens in another window, should also be included.



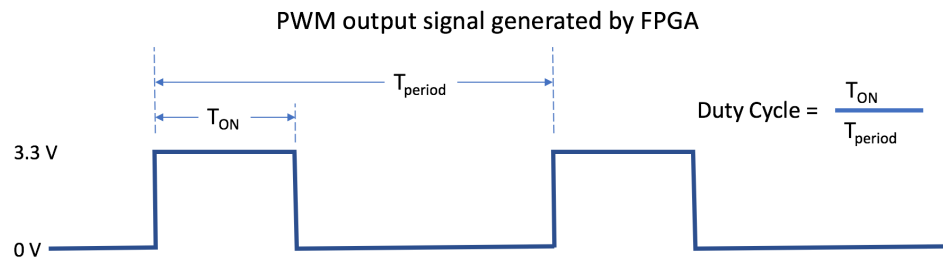
Procedure

1. Prior to the lab, watch the all the lecture videos for this week and the previous week.
2. Create a Lab 6 project with the provided files from D2L. After the project has been setup and the .SV and .XDC files brought into the project, you will need to add the XADC block.
3. Add the XADC IP block from the IP catalogue, to the design. Detailed instructions are given in the Appendix of the Lab 5 XADC and Modularity document, please follow them carefully. It is recommended that you do a quick RTL Schematic, to at least check whether your design looks reasonable, i.e. matches the instructor's RTL Schematic on the previous page. If everything looks fine, generate the Bitstream and download to the Basys3. This will run Synthesis and Implementation, prior to generating the Bitstream.
4. Since the provided design uses a much small `averager` (2^8 versus 2^{12}), this greatly reduces Synthesis times. To further speed up Synthesis times (by almost 2 times, e.g. 14 minutes down to 8 minutes for the instructor's 2^{12} design), you can modify the Synthesis Settings in Vivado (thank you for the great suggestion, Devin Atkin).

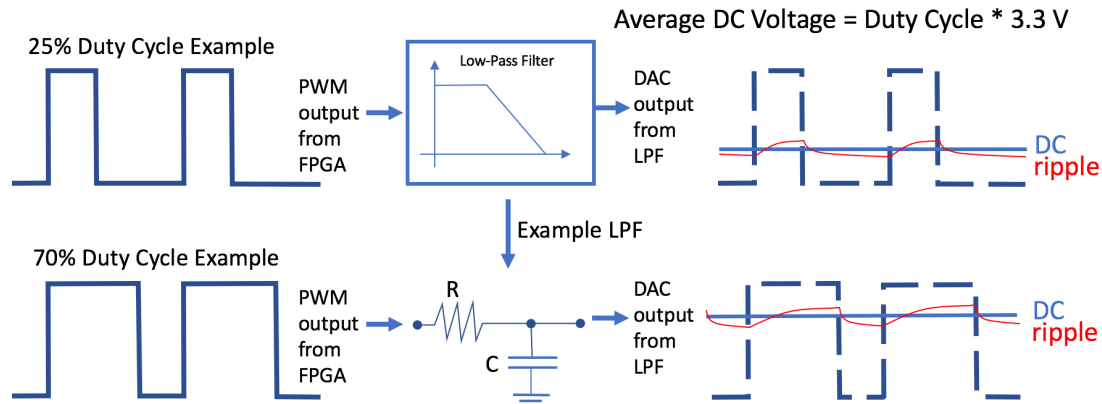
PROJECT MANAGER > Settings > Synthesis > Strategy > Flow_RuntimeOptimized (dropdown). Press OK. This results in a less optimized (bigger, slower) design, but speeds up your design iteration cycles.



5. PWM Basics: A brief description of a PWM DAC operation is given below.



The FPGA output is a square wave with a variable ON time, or T_{ON} . The period of the square wave is T_{period} . The ratio of T_{ON} / T_{period} is the Duty Cycle, and this is the percentage of the period that the square wave is high (e.g. 3.3.V).



In the two examples above, the PWM output from the FPGA is fed into a Low-Pass Filter (LPF) and the LPF can be as simple as an RC filter, as shown in the second example. The output of the LPF is a smoothed signal. This signal has an average DC voltage, which is the 3.3 V supply voltage of the FPGA I/O, scaled by the Duty Cycle. For example, if the PWM waveform had a Duty Cycle of 25%, then the average DC voltage would be:

$$25\% * 3.3 \text{ V} = 0.825 \text{ V.}$$

However, the LPF output signal is not typically a smooth DC value. There is a voltage ripple, as indicated by the red line. The voltage ripple would be the actual voltage seen at the output of the LPF and it depends on the characteristics of the LPF, the PWM period, and the PWM duty cycle.

Recall for an RC low-pass filter, $f_c = 1/2\pi RC$. The low-pass filter should do two functions:

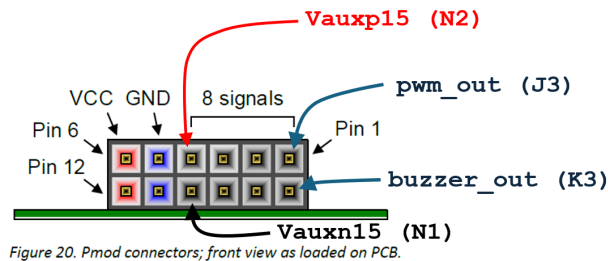
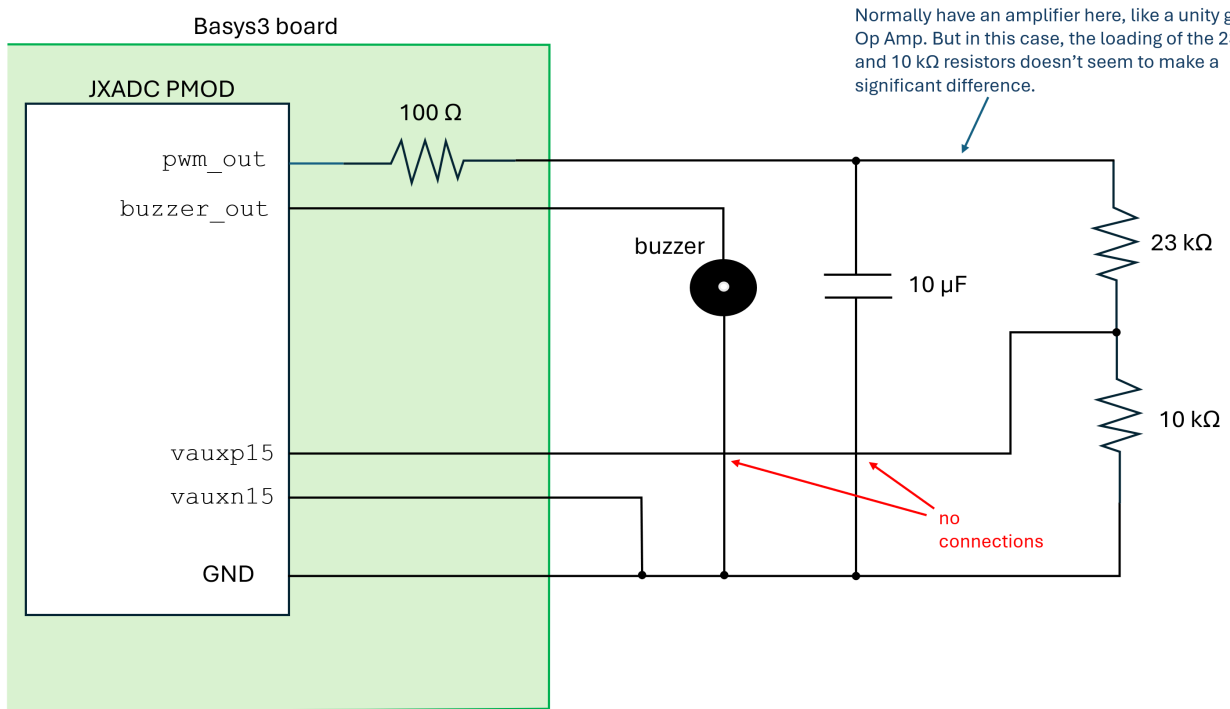
- Be high enough to pass the desired signal, which for us will be a 1 Hz triangle wave.
 - Rule of Thumb: f_c should be around 10x the highest frequency component of the signal, e.g. at least 10 Hz (or **higher**).
- Be low enough to attenuate the PWM switching frequency (i.e. $1/PWM \text{ period}$).
 - Rule of Thumb: f_c should be around 10x lower than the PWM switching frequency.
 - Our clock frequency is 100 MHz and our PWM counter is 8-bits ($2^8 = 256$ values). Then the PWM switching frequency is $f_{PWM} = 100 \text{ MHz} / 256 \sim 390 \text{ kHz}$
 - Then, f_c should be around 39 kHz (or **lower**).

Our rules of thumb suggest that $10 \text{ Hz} < f_c < 39 \text{ kHz}$. You may design your own RC filter.

Recall that there is a 100 Ω series resistor in front of every Basys3 I/O. The instructor used a 10 μF capacitor (along with the Basys3 100 Ω resistor) and got satisfactory results. This resulted in an

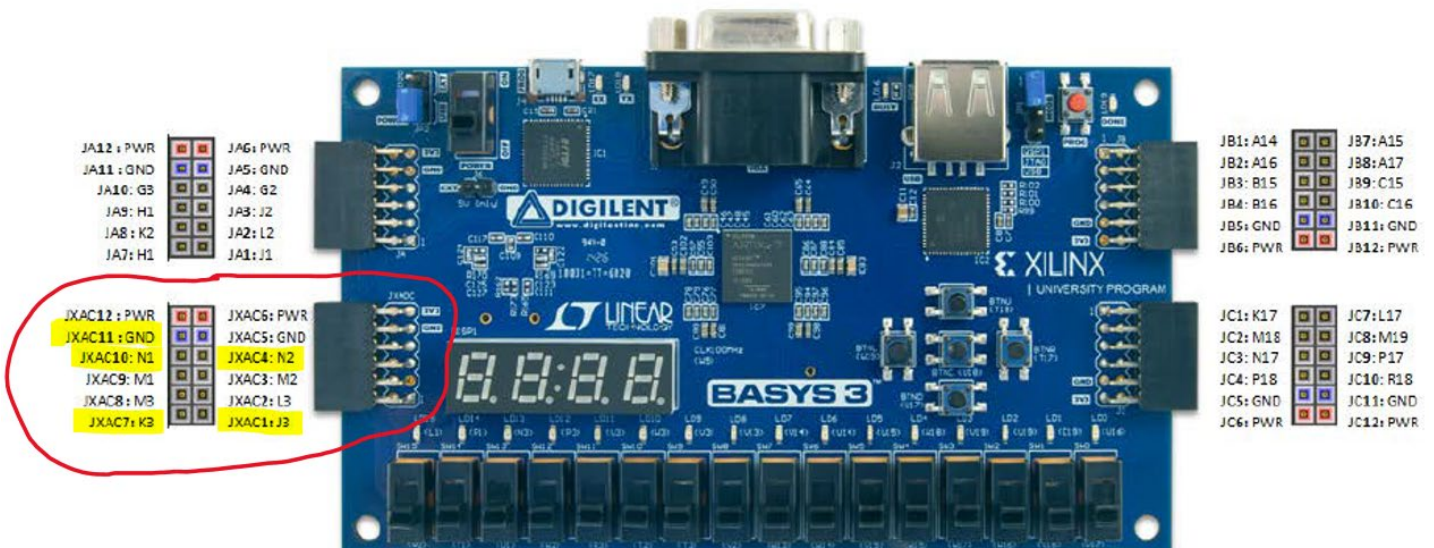
$$f_c = 1/2\pi RC = 1/(2\pi \cdot 100 \Omega \cdot 10 \mu\text{F}) = 159 \text{ Hz.}$$

Below are helpful figures to get you started connecting your Basys3 to your breadboard. Disconnect your Basys3 from the USB, before making connections. When completed, download the Bitstream again and test the `pwm_out` and `buzzer_out`, using the leftmost two slide switches on the Basys3. Examine the FSM module code to understand what slide switch settings are supposed to do what. When operating `pwm_out`, you should see the 7-segment displays count up and down, representing the triangle waveform.



JXADC PMOD

Basys3: Pmod Pin-Out Diagram

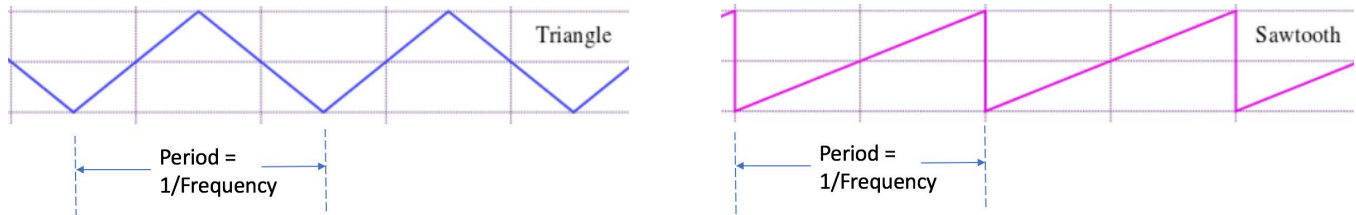


6. Connect the oscilloscope and measure the triangle waveform. Connect the oscilloscope probe across the capacitor and insure that the probe's ground connection goes to the circuit GND. The triangle waveform is set to 1 Hz. It will be easier to see the ADC values counting up and down on the 7-segment displays, if you change the triangle waveform parameter to 0.1 Hz.

DO: copy and paste a snip of your top level RTL Schematic, into your Design Record.

DO: take a picture of the triangle oscilloscope waveform and paste it into your Design Record.

7. Modify the design so that it also produces a sawtooth waveform, as well as a triangle waveform. Ensure that the parameterization of the new module produces the correct frequency, as specified by the frequency parameter. Be sure to also modify the FSM module, and any other necessary changes, to include this operational mode.



DO: take a picture of the sawtooth oscilloscope waveform and paste it into your Design Record.

8. The buzzer produces a 1 kHz tone. Create an additional buzzer module that creates a “chirp” tone. The chirp tone is a one with a constantly varying frequency, say from 500 Hz to 10 kHz, in a short time interval such as two or three seconds. Be sure to also modify the FSM module, and any other necessary changes, to include this operational mode.
9. The LEDs pulse with the triangle waveform. Modify the design so that the 7-segment displays also pulse the same way, with the triangle waveform.

DO: copy and paste a snip of your top level RTL Schematic, into your Design Record.

10. Now, let's figure out how to use the R2R ladder circuit to generate an analog waveform. The R2R ladder is a special voltage divider, where a binary value gets placed from GPIOs (like our Basys3 outputs), onto the R2R ladder. The design already has the R2R ladder outputs coming from the JA PMOD. Figure out from the design and the .XDC constraints file, and the figures below, how to connect the 8 signals from the JA PMOD, to the R2R ladder. If wired correctly, the output of the R2R ladder should be a triangle waveform, just like that of the triangle generated by the PWM output. Consider whether you wish to use a capacitor to smooth the waveform coming out of the R2R ladder.

DO: take a picture of the R2R ladder triangle oscilloscope waveform and paste it into your Design Record.

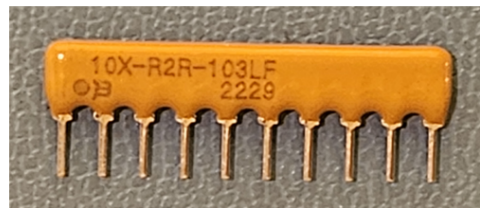
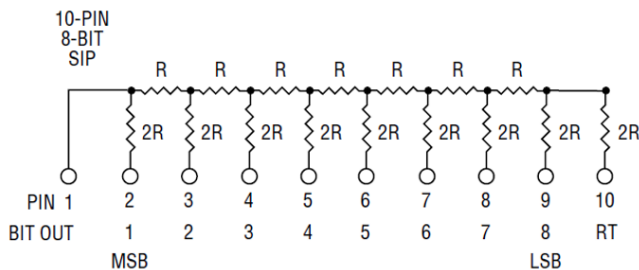
103 means **10** = 10, **3** = 10^3 , so $10 \times 10^3 \Omega$ or 10 k Ω . (e.g. 473 would be 47 k Ω).

In our case for 103:

$$R = 10 \text{ k}\Omega$$

$$2R = 20 \text{ k}\Omega^*$$

***Note:** the Basys3 has 100 Ω series resistors on its I/O's, so the 2R are effectively:
 $20 \text{ k}\Omega + 100 \Omega = 20.1 \text{ k}\Omega$



Pin 1 (BIT OUT): this is the analog output

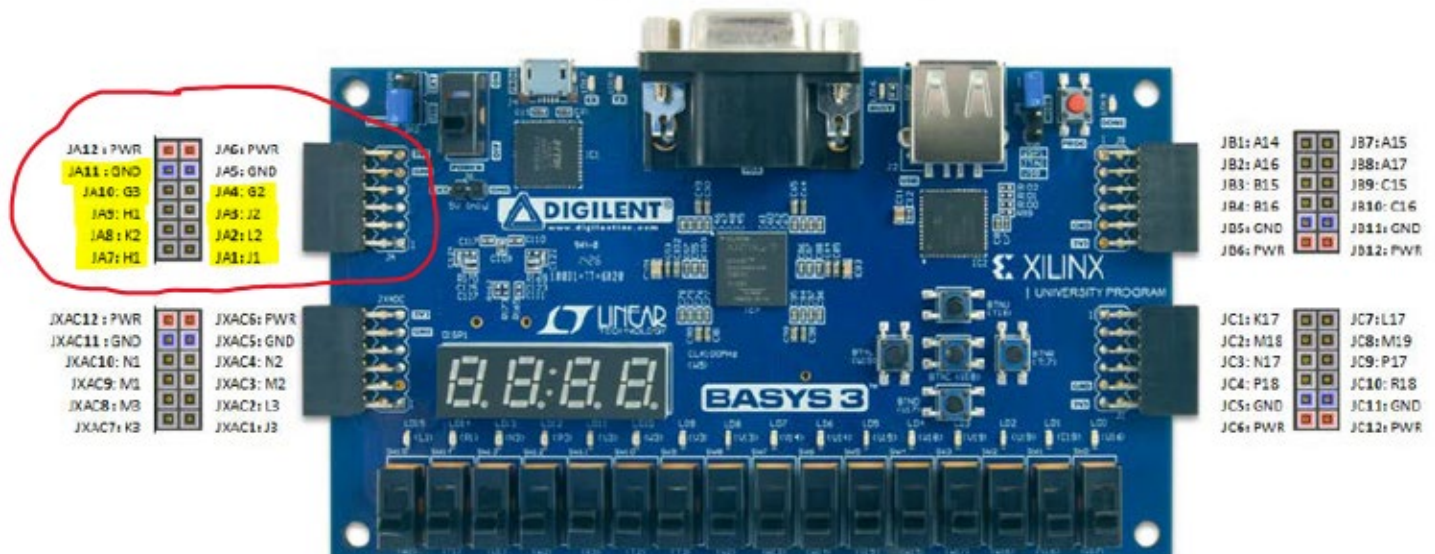
Pin 10 (RT): connect this to Ground (0 V)

The R/2R Ladder Network is commonly used for Digital to Analog (D/A) conversions and Analog to Digital (A/D) conversion by successive approximations. The bits of the ladder are the points at which input signals are presented to the ladder and the output terminal (OUT) is the point at which the output is taken from the R/2R ladder. This terminal (OUT) is commonly used to drive an operational amplifier. R_T (the terminating resistor) is always connected to ground.

Standard R/2R Ladder Networks have a resistance tolerance of $\pm 2.0\%$ ($\pm 1.0\%$ available on all but low profile SIPs).

From: <https://www.bourns.com/docs/technical-documents/technical-library/networks/application-notes/r2rap.pdf>
 and <https://www.bourns.com/docs/Product-Datasheets/R2R.pdf>

Basys3: Pmod Pin-Out Diagram



DO: copy and paste the Project Summary Overview, in your Design Record. Show the calculations for the maximum clock frequency for your design, based on the WNS.

DO: show your working design to your TA.

Project Summary

Device

lab_5_top_level_students.sv

Basys3_Lab_5.xdc

mux4_16_bits.sv

averager.sv

Overview | Dashboard

Settings

Edit

Project name:

Lab_5_students

Project location:

D:/Dropbox/U of C/Teaching/ENEL 453/ENEL 453 F2024/Labs/Lab 5 Students/Lab_5_students

Product family:

Artix-7

Project part:

xc7a35tcpg236-1

Top module name:

lab_5_top_level_students

Target language:

Verilog

Simulator language:

Mixed

Synthesis

Status:

Complete

Messages:

7 warnings

Active run:

synth_1

Part:

xc7a35tcpg236-1

Strategy:

Vivado Synthesis Defaults

Report Strategy:

Vivado Synthesis Default Reports

Incremental synthesis:

Automatically selected checkpoint

Implementation

Sum

Status:

Complete

Messages:

5 warnings

Active run:

impl_1

Part:

xc7a35tcpg236-1

Strategy:

Vivado Implementation Defaults

Report Strategy:

Vivado Implementation Default Reports

Incremental implementation:

None

DRC Violations

Summary:

1 warning

Implemented DRC Report

Timing

Setup

Worst Negative Slack (WNS):

3.379 ns

Total Negative Slack (TNS):

0 ns

Number of Failing Endpoints:

0

Total Number of Endpoints:

12588

Implemented Timing Report

Utilization

Post-Synthesis | Post-Implementation

Graph

Table

LUT

10%

LUTRAM

21%

FF

10%

DSP

1%

IO

32%

BUFG

3%

Utilization (%)

Power

Total On-Chip Power:

0.139 W

Junction Temperature:

25.7 °C

Thermal Margin:

59.3 °C (11.8 W)

Effective θ_{JA} :

5.0 °C/W

Power supplied to off-chip devices:

0 W

Confidence level:

Low

Implemented Power Report

Deliverables

By the end of the lab period or the beginning of the next lab period, demonstrate to the TA:

1. Your Basys3 board running with the latest iteration of the project, be prepared to explain the design and any part of the Vivado design and programming flow.
2. Your Design Record document. Ensure that the Design Record is uploaded to the D2L Dropbox, before seeing the TA.

Your TA may ask any team members at random to answer any questions. Work together to ensure that all team members fully understand the lab project and its deliverables. You may also be asked to demonstrate your ability to proceed through the entire design flow, including:

1. Create and start a new project.
2. Synthesize and download your design to the Basys3.
3. Simulate your design and setup the waveforms.

Rubric

As the term progresses, the expectations will increase as your skills develop. All team members are required to participate in the entire lab period and to present their project to the TA. Missing team members will not receive a mark for the project. The three strikes policy outlined in the Course Outline, will be in effect for all labs.

Points	Criteria
4	Fully complete and high quality, questions answered well.
3	Fully complete and high quality, some questions not answered well or had to have other team members answer.
2	Mostly complete or answers are weakly answered by team members
0	Below acceptable for credit.

There is no 1 point given. Fractional points, e.g. 2.5, are not given.